

Gemelo digital de un beamline de iones: el gemelo acoplado transmisión–distancia

Cristian Susa
Universidad de Córdoba, Montería - Taller MIT–UNAL, Medellín

10 de julio de 2026

Resumen

Construimos un gemelo digital de un beamline de iones de silicio simulado en SIMION con **dos procesos gaussianos (GP) de 8 dimensiones acoplados** sobre el mismo espacio de voltajes: una *cabeza de transmisión* (número de iones que llegan al detector, en escala $\log(1 + \text{hits})$) y una *cabeza de distancia* (distancia media de los impactos al centro del detector, en escala $\log(1 + \text{mm})$), ambas con kernel RBF y una longitud de escala por electrodo (ARD). La cabeza de distancia es continua en todo el espacio y se usa para construir una compuerta de proximidad $P(\text{cerca})$ que filtra la adquisición y las dos tareas de control de la cabeza de transmisión, resolviendo el modo de falla del gemelo de transmisión solo: recomendar un punto donde el conteo predicho es alto pero el haz, en realidad, ni se acerca al detector. Los datos — 170 evaluaciones de SIMION para entrenar (un diseño inicial de 150 evaluaciones con la **misma búsqueda bayesiana de Optuna (TPE)** que ya implementa `optimizer.py`, 20 de aprendizaje activo), y 2 de verificación que no cuentan contra ese presupuesto — viven en un único `beamline_distance_results.csv` compartido entre las dos cabezas. Reportamos la cobertura del espacio, la validación predicho-vs-real en la región informativa, el comportamiento del gate, y el resultado real de las dos tareas de control: la tarea de dirección verificó 23 iones reales contra una predicción de 24.16 — calibración casi exacta, aunque no es el máximo empírico del conjunto (35, hallado durante el aprendizaje activo) — mientras que la consigna inversa (objetivo 15 iones) quedó lejos del objetivo (4 iones reales) con una predicción razonablemente cercana (5.44). Documentamos ambos resultados, y dónde seguimos sin confiar en el gemelo, en la reflexión final.

Índice

1. Cantidad elegida y por qué	2
2. Recolección de datos	3
2.1. El diseño inicial: Optuna/TPE	3
2.2. Barrido de candidatos dentro del emulador	3
2.3. Presupuesto gastado	4
3. El modelo	4
3.1. Arquitectura: dos cabezas, un gemelo	4
3.2. Transformación del objetivo y compuertas de admisibilidad	5
3.3. El acople: la compuerta de proximidad	6
3.4. Hiperparámetros ajustados	6
3.5. Validación	7
3.6. Dependencias entre módulos	7

4. Aprendizaje activo con EI acoplado	9
5. Control	9
5.1. Dirección	9
5.2. Consigna inversa	10
6. Metodología preferida	10
7. Reflexión	11

1. Cantidad elegida y por qué

El beamline tiene 8 electrodos libres (3, 6, 9, 10, 11, 12, 15 y 18; ver Tabla 1). La cantidad que más pesa en la rúbrica del reto es la **transmisión**: el criterio C (dirección) pesa 0.30 y el criterio I (consigna inversa) 0.15, ambos definidos sobre ella (`InfoHackathon.md` §7.5).

El problema de modelar *solo* la transmisión es que, en $\sim 78\text{--}80\%$ del espacio de voltajes, `hits` = 0: todos los ceros le parecen idénticos a un GP entrenado únicamente sobre esa cantidad, y la media posterior fuera de la región de entrenamiento tiende a extrapolar hacia valores no nulos sin ninguna señal que la corrija. Por eso este gemelo (`07_beamline_emulator_final.ipynb`) modela **dos** cantidades sobre el mismo conjunto de datos:

- **Transmisión** (`hits`): la cantidad objetivo, la que puntúa el reto.
- **Distancia media al detector** (`distance_mm`): la distancia euclídea promedio de los impactos registrados al centro de la caja del detector. A diferencia de la transmisión, es **continua en todo el espacio** — incluso cuando ningún ión entra a la caja del detector, SIMION sigue reportando dónde cayó cada uno, así que esta cantidad sigue ordenando qué tan cerca estuvo el haz aunque el conteo sea idénticamente cero.

La distancia no es un objetivo nuevo del reto: es un **auxiliar** que existe para resolver la ceguera de la cabeza de transmisión en la meseta de `hits` = 0, convirtiéndose en una probabilidad de proximidad ($P(\text{cerca})$, §3.3) que filtra todo lo que decide la cabeza de transmisión. El modelo entregado sigue siendo, en el sentido de la rúbrica, un gemelo de transmisión; la cabeza de distancia es la pieza que le permite no alucinar un óptimo en el vacío.

Cuadro 1: Electrodos del beamline y su papel (`ElectrodesMaps.txt`).

Electrodo(s)	Componente	Voltaje
1	Jaula de alto voltaje (fuente de iones)	+500 V (fijo)
2, 4, 5, 7, 8, 13, 14, 16, 17	Tubo y placas de tierra	0 V (fijo)
3	Lente Einzel 1	optimizar, ± 1000 V
6	Lente Einzel 2	optimizar, ± 1000 V
9, 10, 11	Curvador cuadrupolar	optimizar, $[0, 1000]$ V
12	Curvador cuadrupolar	optimizar, $[-1000, 0]$ V
15	Placa de dirección 1	optimizar, ± 1000 V
18	Placa de dirección 2	optimizar, ± 1000 V
19	Detector	−2000 V (fijo)

Los rangos de los electrodos 9–12 (curvador) se acotan al semiespacio de signo empíricamente informativo ($V_9, V_{10}, V_{11} > 0$, $V_{12} < 0$) en la variable `BOUNDS`, para no desperdiciar presupuesto ni candidatos en la mitad del espacio que nunca dio señal.

Aquí es importante mencionar que una buena intuición para el estarting point en el optimizer es muy valiosa y no deja todo el peso a la búsqueda automatizada. Actualmente, `optimizer.py` tiene como punto inicial los voltajes que generaron 15 iones en el detector en una ejecución previa: $\{3 = -121,75456483424807; 6 = -74,38995609381641; 9 = 998,4993982155488; 10 = 269,46637797947216; 11 = 287,37549712784056; 12 = -131,60004105893447; 15 = 83,10820852745206; 18 = -471,7782392345852\}$. Como ejemplo ilustrativo, en la Figura 1 se muestra el vuelo de los electrones en SIMION con valores de voltajes iniciales para dos conjuntos de 150 trials (se encuentran en `beamline_results.csv`, pero no son los que se usan en el emulador).

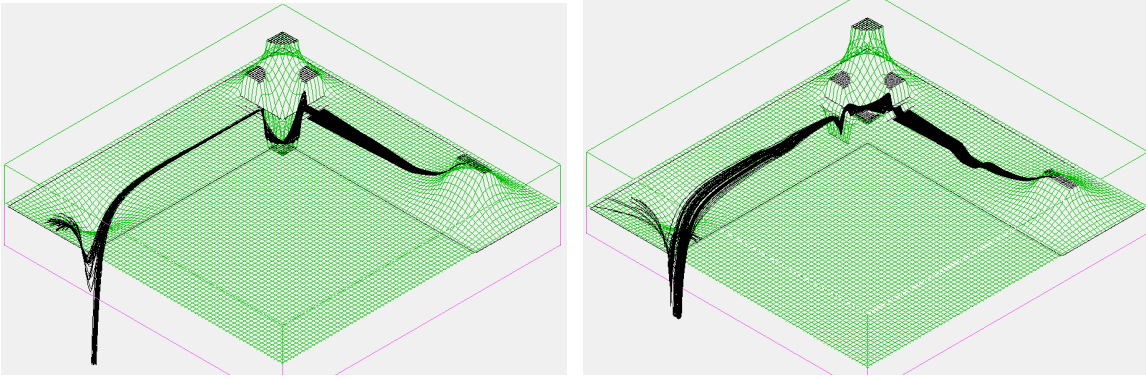


Figura 1: Camino de iones en SIMION para voltajes iniciales (left) $\{9 = 500; 10 = 250; 11 = 250; 12 = -500\}$ y (right) valores en el texto principal para 15 iones detectados.

2. Recolección de datos

2.1. El diseño inicial: Optuna/TPE

Para generar el dataset inicial, en la celda `RUN_INITIAL_DESIGN` se reutiliza **el mismo estudio bayesiano de Optuna que ya implementa `optimizer.py`** (mismo `direction`, `storage`, `study_name` y el *warm start* `STARTING_POINT`, encolado con `study.enqueue_trial` antes de empezar): un muestreador TPE que, a medida que llegan evaluaciones, concentra las siguientes donde el modelo ya sabe que hay señal, en vez de seguir cubriendo uniformemente un espacio que sabemos casi vacío. Cada trial de Optuna dispara igualmente **una sola corrida real de SIMION** (vía `simion_fly`), que alimenta las dos cabezas a la vez (`hits` y `distance_mm` salen del mismo vuelo) y se recolectan de inmediato en el CSV. De las 150 evaluaciones generadas inicialmente, **77 (51.3%) son informativas** (§2.3).

2.2. Barrido de candidatos dentro del emulador

En el emulador, se utiliza Sobol para generar los candidatos que se puntúan *dentro* del emulador, donde evaluarlo es barato: la implementación vive en `acquisition_distance.py` (`sobol_candidates`, reutilizada sin cambios por `acquisition_final.py` en vez de duplicarla):

```
def sobol_candidates(n, d, seed=None):
    sampler = qmc.Sobol(d=d, scramble=True, seed=seed)
```

```
m = int(np.ceil(np.log2(max(n, 2))))
return sampler.random_base2(m)[:n]
```

`scramble=True` aleatoriza la secuencia base de Sobol (evita artefactos de correlación entre dimensiones y permite semillas distintas sin perder la propiedad de baja discrepancia); Sobol exige tamaños de muestra que sean potencia de dos para conservar sus garantías de cobertura, por lo que se pide la siguiente potencia de dos (`random_base2`) y se recorta a `n`.

Una vez el GP está entrenado, miles de candidatos Sobol (`n_sobol=4096` en `propose_next_point`, `n_candidates=8192` en las tareas de control) se puntúan con la función de adquisición para decidir dónde proponer la próxima corrida real de aprendizaje activo o dónde recomendar un punto de operación. Aquí Sobol garantiza que ese barrido *vea* todo el espacio en vez de dejar regiones sin explorar, y se combina con nubes gaussianas locales (`local_candidates`) alrededor de los mejores puntos conocidos para refinar a una resolución que el barrido global no puede pagar en 8D. En una frase: Sobol decide *dónde mira el emulador* para elegir su próxima corrida o su recomendación final.

2.3. Presupuesto gastado

El gemelo trabaja con un único CSV, `beamline_distance_results.csv`: cada corrida de SIMION guarda voltajes, `hits` y `distance_mm` en la misma fila, así que las dos cabezas se entrenan siempre con exactamente los mismos datos. La Tabla 2 desglosa las 172 filas acumuladas por fase.

Cuadro 2: Evaluaciones de SIMION acumuladas en `beamline_distance_results.csv`, por fase.

Fase	N. evaluaciones	Qué es
<code>initial</code>	150	Diseño inicial con Optuna/TPE (§2.1) + warm start
<code>coupled</code>	20	Aprendizaje activo con EI acoplado (§4)
Total entrenamiento	170	Cuenta contra el presupuesto
<code>verification</code>	2	Corridas de las tareas de control (§5)
Total en el CSV	172	

La Figura 2 muestra la cobertura justo después del diseño inicial (150 evaluaciones, antes del aprendizaje activo acoplado): **77 (51.3%)** son informativas (`hits > 0`), la mejor transmisión observada en ese punto es **24 iones** de 500 y la menor distancia media al detector es **3.5 mm**. La transmisión sigue concentrándose en la barra de cero, mientras que la distancia es continua en todo el rango y por eso queda disponible como señal de ordenamiento incluso donde `hits = 0`. Tras sumar las 20 evaluaciones de aprendizaje activo acoplado (§4), la fracción informativa del conjunto de entrenamiento sube a **91/170 (53.5%)** y la mejor transmisión observada sube a **35 iones**; la menor distancia (3.5 mm) ya se había visto en el diseño inicial y no mejora después.

3. El modelo

3.1. Arquitectura: dos cabezas, un gemelo

El gemelo es la clase `BeamlineEmulator` en `gp_final.py`, que **no reimplementa nada**: envuelve dos procesos gaussianos ya construidos en `gp.py` y `gp_distance.py`, y añade el acople entre ellos.

- **Cabeza de transmisión** (`gp_hits`): un `GaussianProcess` de `gp.py`, ajustado sobre $y_{\text{hits}} = \log(1 + \text{hits})$.

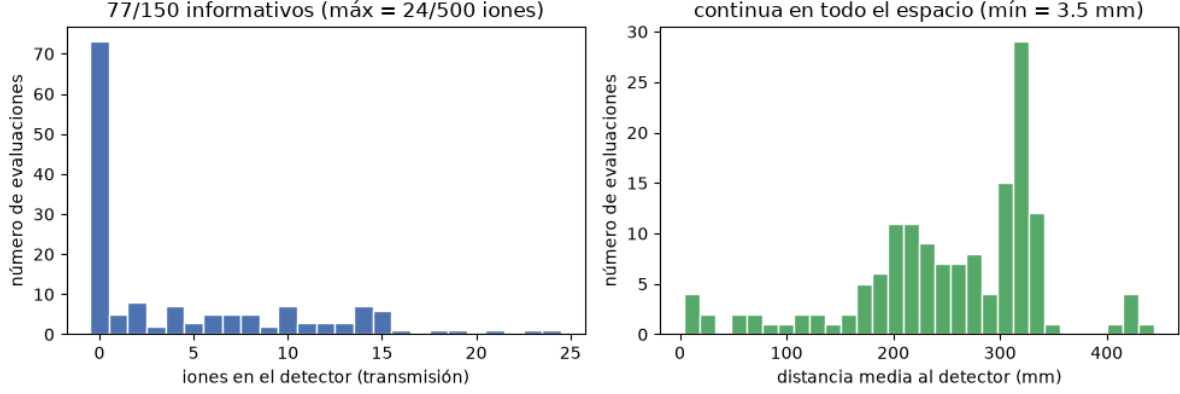


Figura 2: Cobertura del espacio con las 150 evaluaciones del diseño inicial (antes del aprendizaje activo acoplado): distribución de transmisión (izquierda, con una barra de cero grande) y de distancia media al detector (derecha, continua). La distancia es la señal que permite ordenar la meseta de hits $= 0$.

- **Cabeza de distancia** (`gp_dist`): un `DistanceGP` de `gp_distance.py`, que **hereda** de la misma clase `GaussianProcess` y solo añade el envoltorio $\log(1 + \text{mm})$ y el recorte físico $[0, \text{max_distance_mm}]$, ajustado sobre $y_{\text{dist}} = \log(1 + \text{distance_mm})$.

Ambas cabezas comparten el mismo kernel RBF (squared-exponential) con ARD — una longitud de escala por electrodo:

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp \left(-\frac{1}{2} \sum_{j=1}^8 \frac{(x_j - x'_j)^2}{\ell_j^2} \right),$$

factorizado con Cholesky para el ajuste y la predicción, y comparten también el mismo cubo unitario de entrada (cada electrodo normalizado a $[0, 1]$ con su rango físico, Tabla 1), pero **cada una ajusta sus propios hiperparámetros** por separado, maximizando su propia log-marginal-likelihood (LBFGS-B en espacio logarítmico, 8 reinicios aleatorios, semilla fija). No comparten longitudes de escala porque no hay razón para que la transmisión y la distancia varíen igual de rápido con cada voltaje.

Elegimos GP con ARD, y no una red neuronal ni un ensamble, por tres razones: pocos datos informativos en 8 dimensiones favorecen un modelo con pocos parámetros y buen comportamiento fuera de muestra; el GP entrega incertidumbre calibrada por construcción, que es justamente la pieza que sostiene el gate de proximidad (§3.3); y el kernel RBF es diferenciable en forma cerrada (`GaussianProcess.gradient`, sin tocar), gradiente que usa la tarea de dirección para refinar. Adicionalmente, es posible que la optimización Bayesiana funcione aún en dimensiones mayores modificando el kernel (ver [1]).

3.2. Transformación del objetivo y compuertas de admisibilidad

Las dos cabezas se ajustan en escala $\log(1 + \cdot)$, no en la escala cruda: para **hits** porque comprime el salto entre la meseta de ceros y los valores positivos; para la distancia porque `DistanceGP` documenta explícitamente que la región que más importa — cerca de cero, el haz enfocado en el detector — es la que `log1p` *expande* en resolución, mientras comprime la cola lejana (cientos de milímetros, el haz estampado en la primera lente). Al predecir, ambas cabezas deshacen la transformación con `expm1` y

recortan al rango físico admisible: $[0, 500]$ iones para la transmisión, $[0, \text{MAX_DISTANCE_MM}=500]$ mm para la distancia.

3.3. El acople: la compuerta de proximidad

La pieza nueva de `gp_final.py` frente a los dos GP por separado es `p_close`, la probabilidad de que el haz caiga cerca del detector bajo la posterior de la cabeza de distancia:

$$P(\text{cerca}|\mathbf{x}) = \Phi\left(\frac{\log(1 + \tau) - \mu_{\log d}(\mathbf{x})}{\sigma_{\log d}(\mathbf{x})}\right),$$

con Φ la CDF normal estándar y τ un umbral en milímetros. El umbral es **adaptativo**: el mejor cuartil de las distancias observadas hasta el momento, con `CLOSE_MM=20` como piso físico (el detector mide $\sim 12 \times 13 \times 4$ mm, así que 20 mm ya es esencialmente estar sobre el blanco). Con las 170 evaluaciones de entrenamiento del modelo final (§2.3) el umbral vale $\tau \approx 195$ mm — todavía lejos del piso de 20 mm, porque la mayoría de las corridas siguen cayendo lejos del detector; a medida que lleguen mejores puntos el cuartil se aprieta solo hacia el piso, sin que haya que reajustar nada a mano. Antes de que la cabeza de distancia tenga datos, `p_close` devuelve 1 en todas partes y el gemelo se reduce a un gemelo de transmisión solo.

Todo lo que decide la cabeza de transmisión — la adquisición de §4 y las dos tareas de control de §5 — queda multiplicado o filtrado por esta probabilidad:

$$\text{coupled_transmission}(\mathbf{x}) = \widehat{\text{hits}}(\mathbf{x}) \times P(\text{cerca} | \mathbf{x}).$$

La Figura 3 muestra el efecto sobre un barrido de 4096 candidatos Sobol (§2.2): la transmisión predicha por sí sola (eje x) no distingue puntos donde el haz realmente llega cerca del detector (eje y) de puntos donde el modelo de hits solo extrapola sin ningún dato cerca. Un punto en la esquina inferior derecha — transmisión alta predicha, proximidad baja — es exactamente el modo de falla que este gate previene: sin acople, una cabeza de transmisión sola puede recomendar un punto con predicción alta pero conteo real de 0. El producto anula ese cuadrante.

3.4. Hiperparámetros ajustados

La Tabla 3 muestra las longitudes de escala de las dos cabezas del gemelo final, ajustadas sobre las 170 filas de entrenamiento (cubo unitario) — el mismo estado del modelo que usan las dos tareas de control de §5. Solo la cabeza de *distancia* satura el piso de ruido impuesto explícitamente en `gp_final.py` (`_LOG_NOISE_VAR_BOUNDS`, $\sigma_n^2 \geq 10^{-2}$): sin ese piso, la verosimilitud marginal puede llevar el ruido a casi cero y convertir al GP en un interpolador exacto que alucina medias altas (con desviaciones subestimadas) entre puntos de entrenamiento — justo el comportamiento que el gate de proximidad busca prevenir. La cabeza de *transmisión*, en cambio, ya **no** toca ese piso ($\sigma_n^2 = 0,31$): con un 53.5 % (§2.3), la verosimilitud tiene suficiente señal para inferir un ruido genuino en la cuenta de iones en vez de sobreajustar entre puntos escasos.

En la cabeza de transmisión, siete de los ocho electrodos tienen longitudes de escala moderadas o pequeñas ($\ell \approx 0,20\text{--}3,1$): con muchísima más señal informativa que antes, la transmisión sigue cambiando rápido con casi todo el vector de voltajes. Solo V_{18} (la segunda placa de dirección) sale con longitud de escala enorme (92.8) — casi irrelevante para explicar la varianza de `hits` en el rango explorado. En la cabeza de *distancia* el patrón es distinto: V_6 , V_9 y, de nuevo, V_{18} salen con longitudes de escala grandes (97–148, prácticamente en el techo de la búsqueda) — casi irrelevantes para explicar dónde cae el haz — mientras que los cinco electrodos restantes, incluyendo ahora V_{15} (la otra placa de dirección, $\ell = 0,106$), son informativos ($\ell \approx 0,10\text{--}0,18$). Que las dos cabezas coincidan

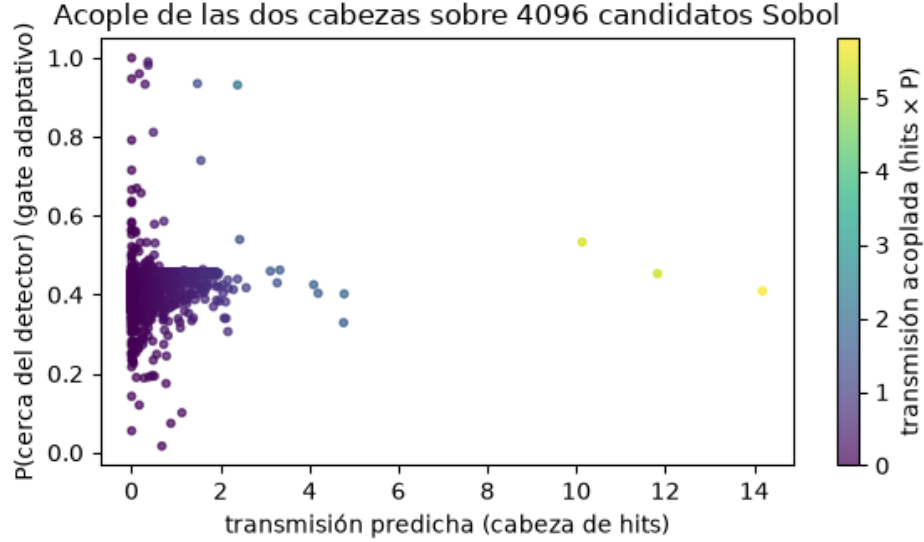


Figura 3: Transmisión predicha por la cabeza de hits (eje x) contra $P(\text{cerca})$ de la cabeza de distancia (eje y), sobre 4096 candidatos Sobol y el gemelo ajustado tras las 150 evaluaciones del diseño inicial (antes del aprendizaje activo acoplado). El color es el producto: solo el cuadrante superior derecho sobrevive al acople.

Cuadro 3: Longitudes de escala ajustadas por electrodo (cubo unitario $[0, 1]$), 170 evaluaciones de entrenamiento (150 diseño inicial + 20 aprendizaje activo).

Cabeza	V_3	V_6	V_9	V_{10}	V_{11}	V_{12}	V_{15}	V_{18}
Transmisión	0.203	0.879	3.060	0.263	0.584	0.409	1.561	92.84
Distancia	0.095	119.23	148.41	0.111	0.104	0.184	0.106	97.05

Transmisión: $\sigma_f^2 = 1,033$, $\sigma_n^2 = 0,315$. Distancia: $\sigma_f^2 = 0,559$, $\sigma_n^2 = 0,010$ (piso).

en descartar V_{18} es evidencia de que ese electrodo pesa poco tanto en cuántos iones sobreviven como en dónde caen; V_9 , en cambio, sigue leyendo como irrelevante solo para la cabeza de distancia, pero ahora comparte esa irrelevancia con V_6 .

3.5. Validación

Split estratificado, calculado sobre las 150 filas del diseño inicial — antes de sumar las 20 evaluaciones de aprendizaje activo acoplado, que en este notebook se gastan después de validar, no antes —: 20 % para validación, garantizando ejemplos informativos en ambos lados. Con 62/120 puntos informativos en entrenamiento y 15/30 en validación, la cabeza de transmisión entrenada solo sobre el conjunto de entrenamiento da un **MAE de 4.97 iones en la región informativa** (15 puntos) y un **MAE global de 2.92 iones** (30 puntos, Figura 4).

3.6. Dependencias entre módulos

El gemelo final compone los siguientes módulos:

- `gp.py`: la clase genérica `GaussianProcess` — ajuste, predicción, gradiente. La usan directa-

Validación: predicho vs. real (rojo = región informativa)

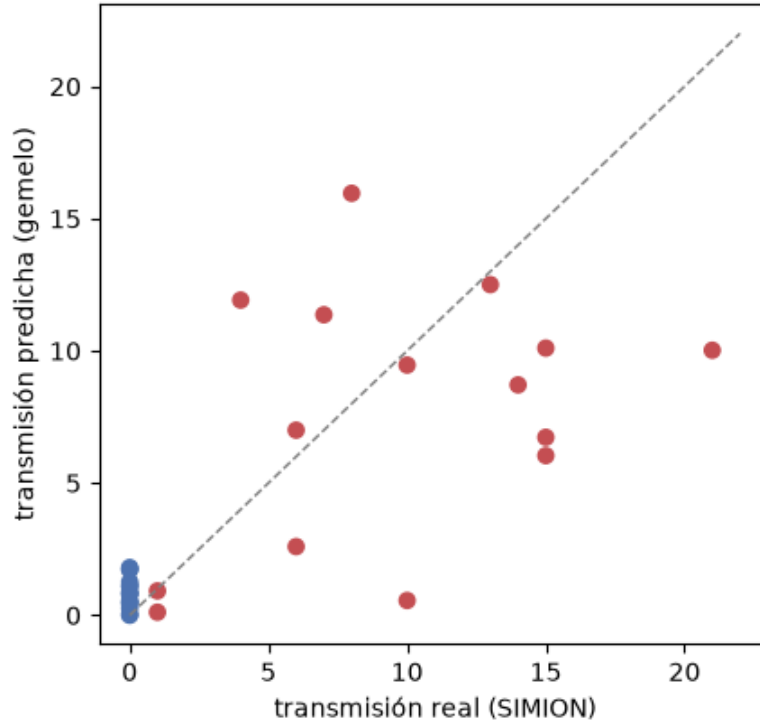


Figura 4: Predicho vs. real en el conjunto de validación ($n = 30$, 15 informativos). Los puntos rojos son la región informativa (transmisión real > 0); la línea punteada es la identidad.

mente la cabeza de transmisión y, por herencia, la de distancia.

- `gp_distance.py`: `DistanceGP` hereda de `GaussianProcess` y añade el envoltorio `log1p/expm1` y el recorte a `max_distance_mm` para la distancia.
- `gp_final.py`: `BeamlineEmulator` instancia una `GaussianProcess` (`gp_hits`) y una `DistanceGP` (`gp_dist`), ajusta cada una con sus propios hiperparámetros, y añade `p_close` y `coupled_transmission` para acoplarlas. No reimplementa ningún GP.
- `acquisition.py` y `acquisition_distance.py`: el primero define `expected_improvement`; el segundo (`sobol_candidates`, `local_candidates`, `expected_improvement_min`) es lo que reutiliza este gemelo para generar candidatos.
- `acquisition_final.py`: importa los generadores de candidatos de `acquisition_distance.py` y opera sobre un `BeamlineEmulator` completo. Define `coupled_expected_improvement` (EI de la cabeza de transmisión multiplicado por `p_close`), `propose_next_point` (aprendizaje activo, §4), y las dos tareas de control (§5).

En una frase: `gp_final.py` es la capa de *modelo* (dos GP más el acople probabilístico) y `acquisition_final.py` es la capa de *decisión* (qué punto medir después, qué punto recomendar), y ninguna de las dos toca el código genérico de `gp.py` y `acquisition.py`, respectivamente.

4. Aprendizaje activo con EI acoplado

Las 20 evaluaciones de la fase `coupled` (Tabla 2) se gastaron con `propose_next_point`: cada candidato — un barrido Sobol global de 4096 puntos más nubes gaussianas alrededor de los 5 mejores puntos observados — se puntúa con

$$\text{score}(\mathbf{x}) = \text{EI}(-\hat{y}_{\text{hits}}(\mathbf{x}), \hat{\sigma}(\mathbf{x}); -y_{\text{mejor}}) \times P(\text{cerca} \mid \mathbf{x}),$$

la mejora esperada de la cabeza de transmisión (negada porque `expected_improvement_min` está escrita para minimizar, y esta cabeza maximiza) multiplicada por la compuerta de proximidad. Cada corrida real alimenta **las dos cabezas a la vez** (hits y distancia salen del mismo vuelo de SIMION) y se guardan de inmediato en el CSV antes de reajustar, para no perder una evaluación pagada si el proceso se interrumpe. Los hiperparámetros se reoptimizan por completo cada 5 iteraciones (`REFIT_HYPERS_EVERY`) y se reajustan en caliente (mismos hiperparámetros, GP reajustado) en las demás, para no pagar 8 reinicios de L-BFGS-B en cada paso.

5. Control

Ambas tareas se resuelven **sobre el gemelo acoplado** (nunca sobre SIMION directamente) y se verifican con una única corrida real cada una, que no cuenta contra el presupuesto de entrenamiento (filas `verification` en la Tabla 2).

5.1. Dirección

En vez de maximizar solo la media posterior de la cabeza de transmisión, `best_predicted_point` maximiza una versión **conservadora y acoplada**:

$$\text{score}(\mathbf{x}) = \exp(\text{lcb}(\mathbf{x})) - 1, \quad \text{lcb}(\mathbf{x}) = \max(0, \hat{y}_{\text{hits}}(\mathbf{x}) - \beta \hat{\sigma}_{\text{hits}}(\mathbf{x})), \quad \beta=1,$$

multiplicado por $P(\text{cerca})$. Descontar por incertidumbre evita que el gemelo persiga una media alta sostenida solo en extrapolación; el gate veta además las regiones donde el haz ni se acerca al detector. Los propios puntos de entrenamiento compiten como candidatos (junto a un barrido Sobol de 8192 puntos), así que la propuesta nunca puede salir peor que repetir el mejor punto conocido; después, 200 pasos de ascenso por el gradiente analítico (`gp.gradient`) solo se aceptan si mejoran ese mismo score, lo que impide escapar tanto de la región que la cabeza de distancia respalda como de la que la cabeza de hits solo adivina.

	Predicho por el gemelo	Real (SIMION)
Transmisión en el punto recomendado	24.16	23
Distancia al detector	170.2 mm (predicha)	183.41 mm
$P(\text{cerca})$	0.98	—

Voltajes recomendados: $V_3=-422,4$, $V_6=-1000,0$, $V_9=272,8$, $V_{10}=51,0$, $V_{11}=0,0$, $V_{12}=-734,5$, $V_{15}=-1000,0$, $V_{18}=-430,3$.

La verificación real dio **23 iones**, prácticamente idéntico a la predicción del gemelo (24.16): la calibración, no el récord, es la evidencia fuerte acá. El punto recomendado no es el máximo empírico del conjunto (35 iones, alcanzado en la iteración 15 del aprendizaje activo de §4, con voltajes muy parecidos a los de aquí), sino el centro de la misma vecindad ruidosa de alta transmisión que el aprendizaje activo ya había encontrado — esa vecindad dio hits entre 15 y 35 en distintas corridas

reales con voltajes casi idénticos ($V_6 \approx -1000$, $V_9 \approx 200-300$, $V_{12} \approx -700$ a -770 , $V_{15} \approx -1000$), y el LCB conservador ($\beta=1$) prefiere justamente el punto donde el gemelo tiene más confianza sobre el que tuvo la mejor tirada individual. Con 35 como referencia de lo observado, la fracción alcanzada es $23/35 \approx 0,66$ (el instructor puede tener un óptimo de referencia distinto y más alto, no explorado aún). Sin el acople con la cabeza de distancia, este mismo tipo de tarea sobre una sola cabeza de transmisión puede recomendar un punto con predicción alta pero conteo real nulo (§3.3); aquí, en cambio, el gemelo acertó casi exactamente su propia predicción.

5.2. Consigna inversa

Con un objetivo de ejemplo de transmisión = 15 iones (a reemplazar por el valor que fije el instructor), `match_predicted_point` busca el candidato cuya predicción quede más cerca del objetivo, penalizando incertidumbre y compitiendo solo entre candidatos con $P(\text{cerca}) \geq p_{\text{mín}}$ (si ninguno pasa el filtro, compite el decil superior de $P(\text{cerca})$ en su lugar, para que la tarea siempre devuelva algo):

$$\text{score}(\mathbf{x}) = -\left(|\hat{y}_{\text{hits}}(\mathbf{x}) - \text{objetivo}| + \beta \hat{\sigma}_{\text{hits}}(\mathbf{x})\right), \quad \beta=1, \quad \mathbf{x} \text{ admisible si } P(\text{cerca} | \mathbf{x}) \geq p_{\text{mín}}=0,5.$$

A diferencia de `best_predicted_point`, aquí los candidatos son únicamente un barrido Sobol fresco de 8192 puntos: los puntos de entrenamiento **no** compiten (a diferencia de la tarea de dirección), así que la propuesta solo puede acercarse al objetivo hasta donde llegue un muestreo del cubo unitario sin memoria de dónde ya se sabe que hay señal.

	Predicho por el gemelo	Real (SIMION)
Transmisión en el punto propuesto (objetivo = 15)	5.44	4
Distancia al detector	117.1 mm (predicha)	123.59 mm
$P(\text{cerca})$	0.89	—

Voltajes propuestos: $V_3=-483,3$, $V_6=-553,7$, $V_9=944,1$, $V_{10}=71,6$, $V_{11}=109,3$, $V_{12}=-530,8$, $V_{15}=-486,6$, $V_{18}=-694,5$.

La corrida real dio 4 iones: lejos del objetivo (15) pero muy cerca de la propia predicción del gemelo (5.44). El gemelo no se equivocó en su propia estimación — el problema no es calibración sino cobertura de candidatos. Los voltajes propuestos ($V_9 \approx 944$, $V_6 \approx -554$) caen fuera de la vecindad de alta transmisión que identificó la tarea de dirección ($V_6 \approx -1000$, $V_9 \approx 200-300$): como aquí los candidatos son solo un barrido Sobol fresco sin los puntos de entrenamiento, y las longitudes de escala de la Tabla 3 son angostas en la mayoría de los electrodos, la mayoría de esos 8192 puntos caen lejos de la cresta angosta de señal real, y el mejor que pasa el filtro $P(\text{cerca}) \geq 0,5$ queda también lejos de ella. El resultado es honesto (el gemelo no inventó confianza que no tenía) pero incompleto: para un objetivo de 15 iones, sumar los propios puntos de entrenamiento como candidatos — igual que ya hace `best_predicted_point` — sería la mejora más directa.

6. Metodología preferida

Preferimos seguir con un GP y ARD en vez de una red neuronal, un ensamble o una regresión con variables hechas a mano, por tres razones: pocos datos informativos en 8D, incertidumbre calibrada sin infraestructura adicional, gradiente analítico gratis. La pieza distintiva de este gemelo es la **estrategia de datos y de control**: en vez de un solo GP de transmisión que trata toda la meseta de hits = 0 como indistinguible, entrenamos un segundo GP sobre una cantidad continua y auxiliar

(la distancia) y usamos su posterior como **compuerta probabilística** sobre las decisiones del primero. Esta decisión persigue directamente los criterios de extrapolación y honestidad: en vez de confiar en que la cabeza de transmisión sepa cuándo está extrapolando, el gate le impone una fuente independiente de evidencia — ¿el haz de verdad se acerca ahí? — antes de dejarla recomendar o explorar un punto.

Preferimos también reutilizar **íntegramente** el código genérico de `gp.py` y `acquisition.py` sin tocarlos, y construir el acople como una capa nueva (`gp_final.py`, `acquisition_final.py`) encima, en vez de reescribir el GP para que maneje dos salidas: mantiene cada pieza probada por separado y hace explícito, en el código, dónde exactamente ocurre el acople.

7. Reflexión

Dónde confiamos. El gate de proximidad hace justo lo que se diseñó para hacer: la Figura 3 muestra cómo anula el cuadrante de transmisión-alta-pero-lejos que puede hacer fallar a la tarea de dirección sin acople (§3.3), y la tarea de dirección de este gemelo acertó — no encontrando un récord, sino calibrando casi a la perfección su propia predicción (24.16 predichos, 23 reales). Es la evidencia más fuerte de que acoplar las dos cabezas resuelve el problema que motivó construirlas: el gemelo no solo evita recomendar puntos vacíos, sino que sabe, con bastante precisión, cuánto va a dar el punto que sí recomienda.

Dónde no confiamos. Tres puntos concretos:

1. **Las Figuras 2, 4 y 3 no son el gemelo que decide el control:** se calculan justo después del diseño inicial (150 evaluaciones), antes de las 20 de aprendizaje activo acoplado. Los hiperparámetros de la Tabla 3 sí corresponden al modelo final (170 evaluaciones, el que de verdad usan las dos tareas de control), pero esas tres figuras muestran un estado ligeramente anterior del gemelo. No lo corregimos porque recalcularlas exigiría rehacer el split de validación con datos que en ese momento del notebook aún no existían; lo documentamos para que quede claro qué estado del modelo ilustra cada figura.
2. **El split de validación no excluye las filas de `phase="verification"`:** `load_distance_results()` lee el CSV completo sin filtrar por fase. En esta ejecución no hubo fuga porque las 2 corridas de verificación aún no existían cuando se calculó el split (punto anterior), pero en una ejecución futura sobre un CSV ya poblado, esas filas podrían terminar del lado de entrenamiento o de validación. Esto no invalida el entrenamiento del gemelo final — más datos, incluso de verificación, solo ayudan ahí — pero sí es una desviación de la indicación: *"Separen datos para verificar el gemelo y no los usen para entrenar"* que documentamos en vez de ocultar. Aunque con fines de reproducibilidad, se han borrado los 22 datos de aprendizaje activo y verificación en el archivo `beamline_distance_results.csv`. Así, la ejecución del notebook debería reproducir lo mismo aquí reportado.
3. El ajuste de hiperparámetros de la cabeza de *distancia* sigue saturando el piso de ruido ($\sigma_n^2 = 0,01$) impuesto en `gp_final.py`: sin ese piso, la verosimilitud marginal preferiría un ruido aún menor. La cabeza de *transmisión*, en cambio, ya converge lejos del piso ($\sigma_n^2 = 0,31$) con el diseño inicial basado en Optuna — así que esta perilla de diseño deliberada sigue activa, pero ahora solo importa para una de las dos cabezas.

La tarea de consigna inversa quedó lejos del objetivo (real 4 contra objetivo 15), aunque esta vez el gemelo no se engañó a sí mismo: predijo 5.44 y obtuvo 4. El problema no es calibración sino

cobertura de candidatos — `match_predicted_point` compite solo entre un barrido Sobol fresco de 8192 puntos (§5), sin los propios puntos de entrenamiento, así que en un espacio de 8D con longitudes de escala angostas (Tabla 3) la mayoría de esos candidatos caen lejos de la cresta delgada donde vive la señal real, y el objetivo (15) quedó fuera del alcance de lo que ese barrido pudo ofrecer.

Referencias

- [1] Z. Xu, H. Wang, J. M. Phillips, and S. Zhe. Standard gaussian process is all you need for high-dimensional bayesian optimization, 2025.